ASSIGNMENT-3.1

P.HARSHAVARDHAN

2303A53038
BATCH - 46

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate

a Python function that checks whether a given number is a palindrome.

Prompt:

```
#Write a Python function that takes an integer as input and
checks whether the number is a palindrome, returning True if it
is and False otherwise.
```

Input:

```python
def is_palindrome(number):
    """
    Check if an integer is a palindrome.

    A palindrome number reads the same forwards and backwards.
    Examples: 121, 1331, 7, 12321

    Args:
        number (int): The integer to check

    Returns:
        bool: True if the number is a palindrome, False otherwise
    """
    # Convert number to string for easy comparison
    num_str = str(number)

    # Check if the string is equal to its reverse
    return num_str == num_str[::-1]


# Example usage:
if __name__ == "__main__":
    # Test cases
    print(is_palindrome(121))    # True
    print(is_palindrome(1331))   # True
    print(is_palindrome(123))    # False
    print(is_palindrome(7))      # True
    print(is_palindrome(12321))  # True
```

Output:

```
True
True
False
True
True
```

Observation:
The program correctly checks whether a number is a palindrome. It works properly for normal numbers and single-digit values. Negative numbers are correctly identified as not palindromes.

Question 2: One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and

ask the AI to generate a Python function to compute the factorial of a

given number.

Example:

Input: 5 → Output: 120

Prompt:

```
#Write a Python function factorial(n) that returns the factorial of a non-negative integer
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
    return int_list
n = int(input("Enter a non-negative integer: "))
print(factorial(n))
```

Output:

```
Enter a non-negative integer: 5
120
```

Observation:

The one-shot prompt produced clearer and more accurate code than the zero-shot prompt because the example guided the AI, reduced ambiguity, and ensured correct handling of the base case (n = 0).

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples

to guide the AI in generating a Python function to check whether a

given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Prompt:

```python
#Write a Python function to check whether a given number is an Armstrong number
def is_armstrong(num):
    order = len(str(num))
    sum_of_powers = sum(int(digit) ** order for digit in str(num))
    return num == sum_of_powers
n = int(input("Enter a number: "))
if is_armstrong(n):
    print(f"{n} is an Armstrong number.")
else:
    print(f"{n} is not an Armstrong number.")
```

Output:

```
Enter a number: 153
153 is an Armstrong number.
```

Observation:

Few-shot prompting improved accuracy and logic clarity by guiding the AI with multiple Armstrong number examples.

(Optional Extension)

Question 4: Context-Managed Prompting (Optimized Number

Classification)

Design a context-managed prompt with clear instructions and

constraints to generate an optimized Python program that classifies a

number as prime, composite, or neither.

Prompt:

#Write an optimized Python program that classifies an integer as "Prime", "Composite", or "Neither" with efficient √n logic, clean functions, and no extra variables.

```python
#Write an optimized Python program that classifies an integer as "Prime", "Composite", or "Neither
def classify_number(n):
    if n <= 1:
        return "Neither"
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return "Composite"
    return "Prime"
n = int(input("Enter an integer: "))
result = classify_number(n)
print(f"The number {n} is classified as: {result}")
```

Output:

```
Enter an integer: 12
The number 12 is classified as: Composite
```

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to

generate a Python function that checks whether a given number is a

perfect number.

```python
#Write a Python function that checks if a given integer is a perfect number and returns True or False
def is_perfect_number(n):
    if n <= 1:
        return False
    divisors_sum = sum(i for i in range(1, n) if n % i == 0)
    return divisors_sum == n
n = int(input("Enter a positive integer: "))
if is_perfect_number(n):
    print(f"{n} is a perfect number.")
else:
    print(f"{n} is not a perfect number.")
```

Output:

```
Enter a positive integer: 10
10 is not a perfect number.
```

Observation:

The zero-shot prompt is effective because it clearly defines the task and expected output without examples.

Question 6: Few-Shot Prompting (Even or Odd Classification with

Validation)

Write a few-shot prompt by providing multiple input-output

examples to guide the AI in generating a Python program that

determines whether a given number is even or odd, including proper

input validation.

Prompt:

```python
#Write a Python program that validates input and classifies numbers as even or odd
def classify_even_odd():
    try:
        n = int(input("Enter an integer: "))
        if n % 2 == 0:
            return f"{n} is Even"
        else:
            return f"{n} is Odd"
    except ValueError:
        return "Invalid input. Please enter a valid integer."
print(classify_even_odd())
```

Output:

```
Enter an integer: 8
8 is Even
```

```
Enter an integer: 15
15 is Odd
```

```
Enter an integer: 0
0 is Even
```

Observation:

The few-shot prompt is effective because it guides the model with clear input-output examples and enforces input validation.